



Institución
Universitaria
Reacreditada en Alta Calidad

ESTRUCTURA DE DATOS Y »»» LABORATORIO «««

580506004-8

William Giraldo Escobar
Facultad de Ingenierías
Departamento de Sistemas de Información



Estructura de datos con memoria dinámica

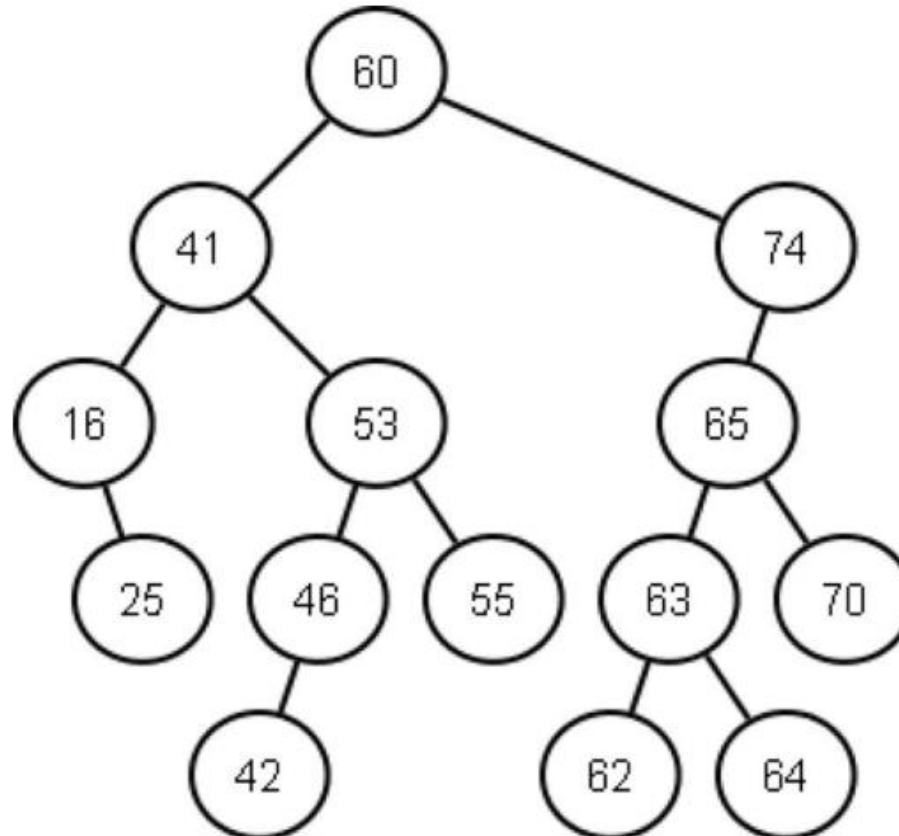
Las estructuras de datos con memoria dinámica se usan para **optimizar** el uso de **memoria** cuando **almacenamos** datos

Estas estructuras siguen siendo **colecciones** de datos, pero con formas de acceso **diferentes**

Las estructuras de datos de memoria dinámica son:

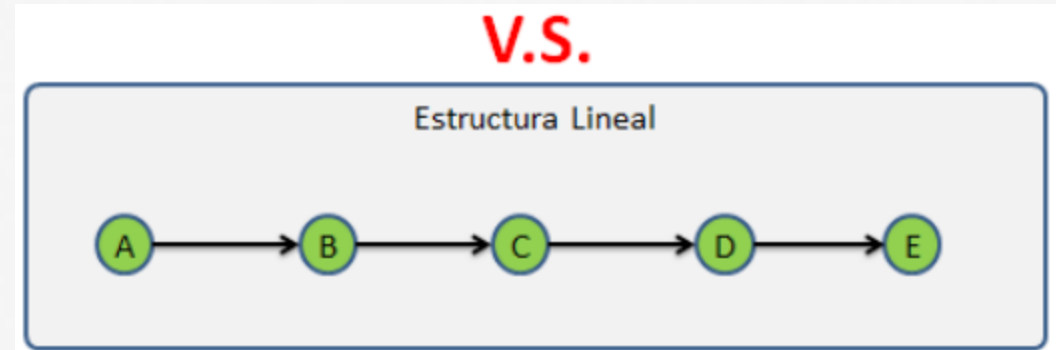
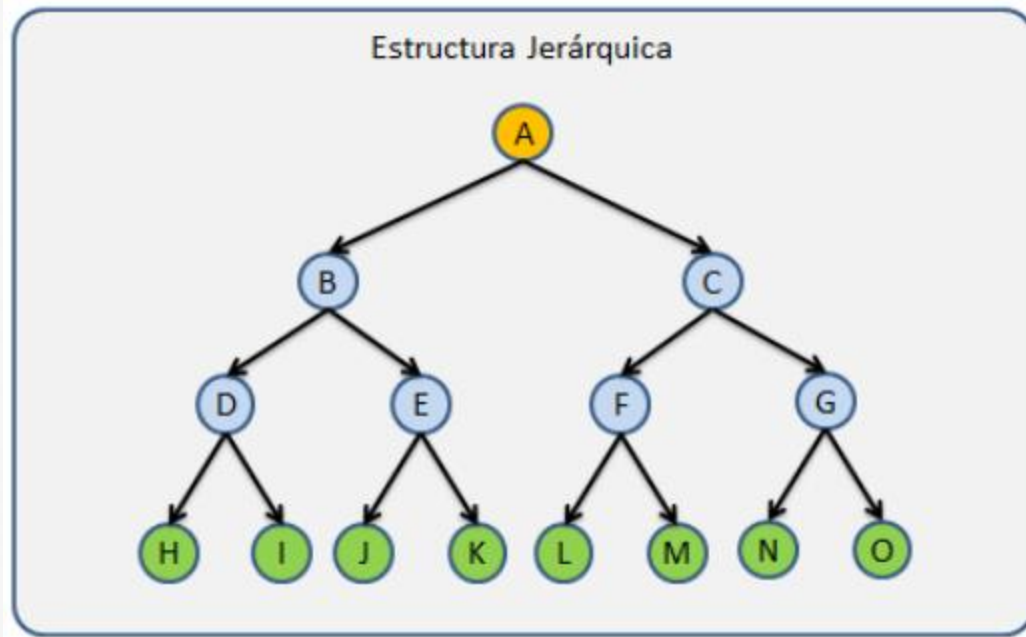
- Listas enlazadas simples
- Listas enlazadas dobles
- Listas circulares
- Pilas
- Colas
- **Árboles**

Es una estructura de datos *jerárquica* y *no lineal* utilizada para ordenar un conjunto de *nodos* donde cada uno almacena un dato.



Los árboles son estructuras dinámicas, porque pueden cambiar tanto de *forma* como de *tamaño* durante la ejecución del programa.

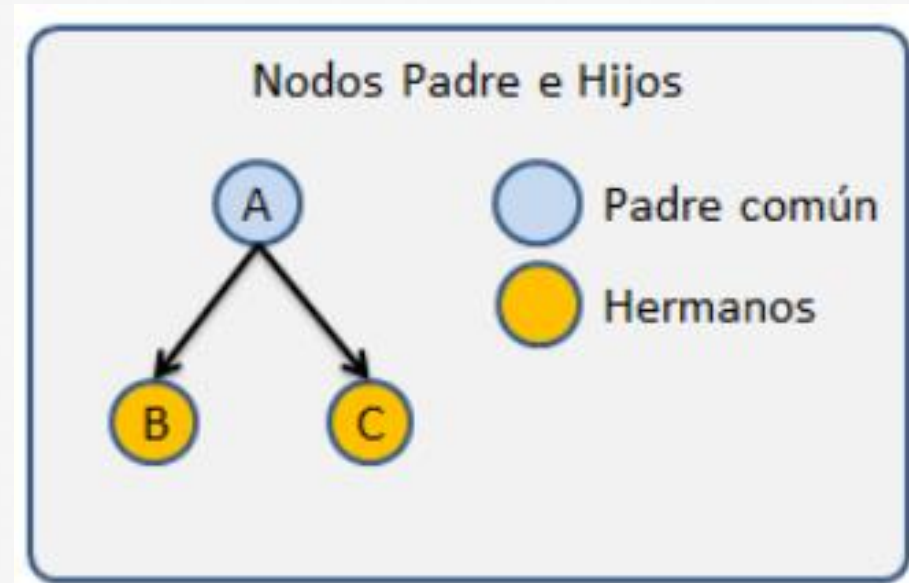
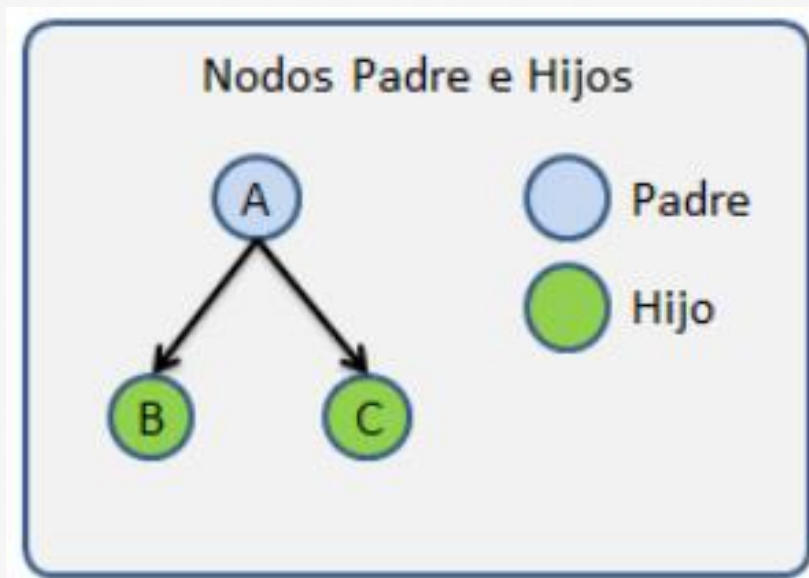
Son estructuras *no lineales* porque cada elemento del árbol puede tener *más de un sucesor*.



ÁRBOLES

relaciones entre nodos

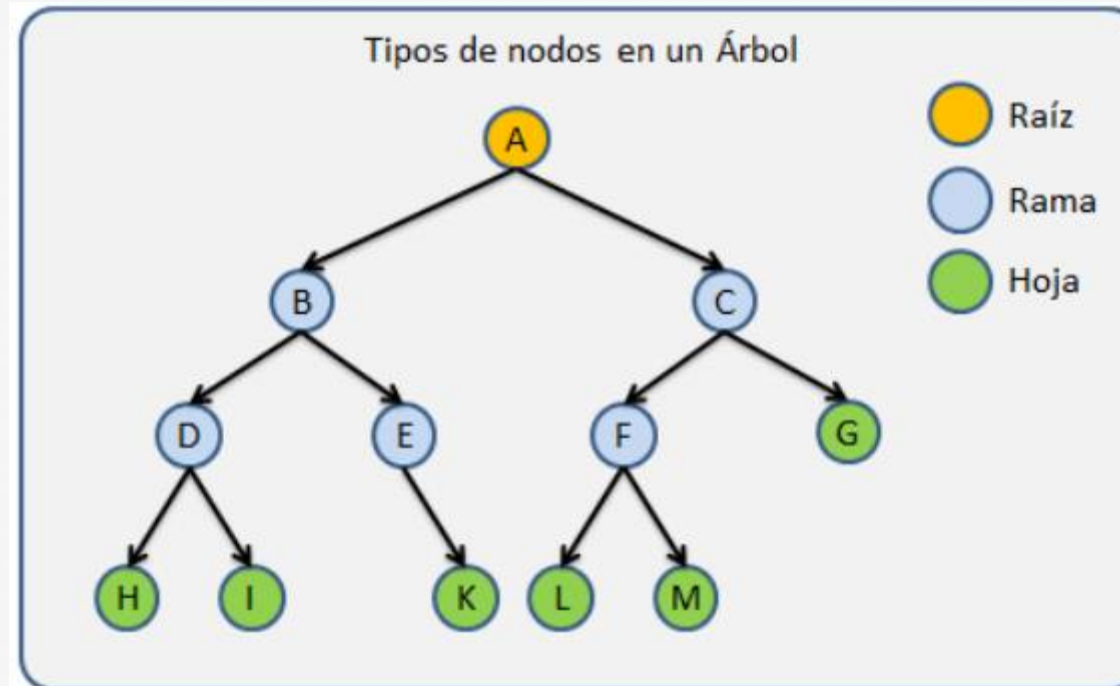
- **Nodo padre:** Se utiliza este término para llamar a todos aquellos nodos que tienen al menos un sucesor.
- **Nodo hijo:** Los hijos son todos aquellos nodos que tienen un padre.
- **Nodo hermano:** Los nodos hermanos son aquellos nodos que comparten un mismo padre en común dentro de la estructura.



ÁRBOLES

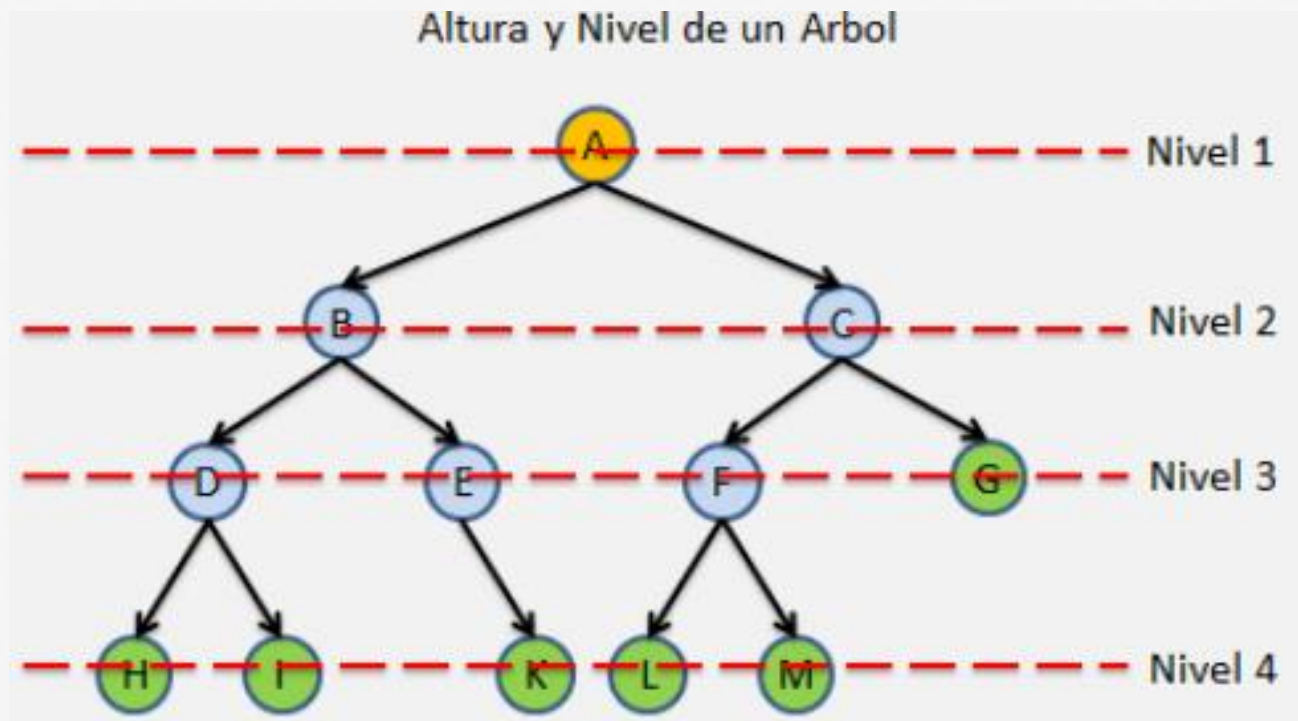
posiciones dentro del árbol

- **Nodo Raíz:** Se refiere al primer nodo de un árbol, Solo un nodo del árbol puede ser la raíz.
- **Nodo Hoja:** Son todos aquellos nodos que no tienen hijos, los cuales siempre se encuentran en los extremos de la estructura.
- **Nodo Interior o Rama:** Estos son todos aquellos nodos que no son la raíz y que además tiene al menos un hijo.



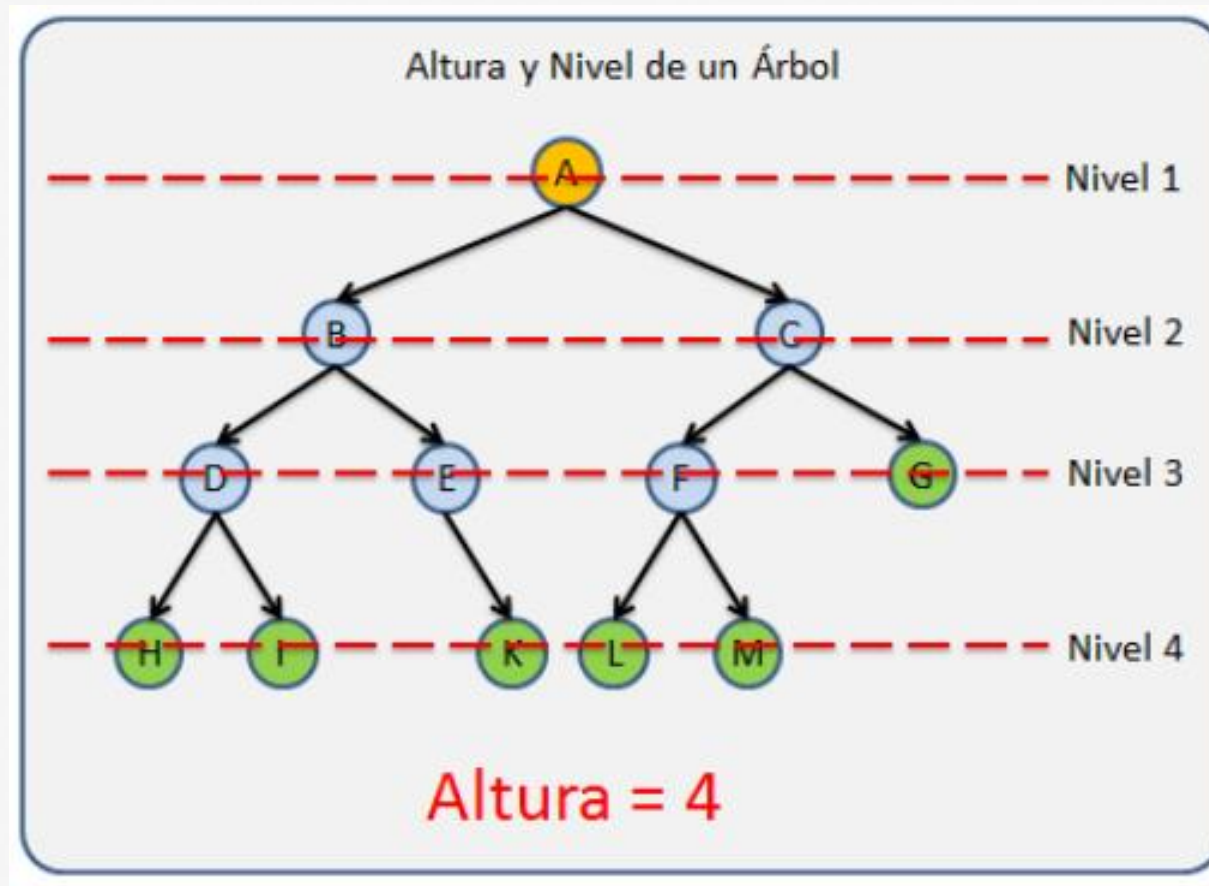
ÁRBOLES - tamaño

- **Nivel:** El nivel de un nodo es su *distancia a la raíz*. Por lo tanto:
 - Un árbol vacío tiene 0 niveles
 - El nivel de la raíz es 1
 - El nivel de cada nodo se calcula contando cuantos nodos existen sobre él, hasta llegar a la raíz + 1



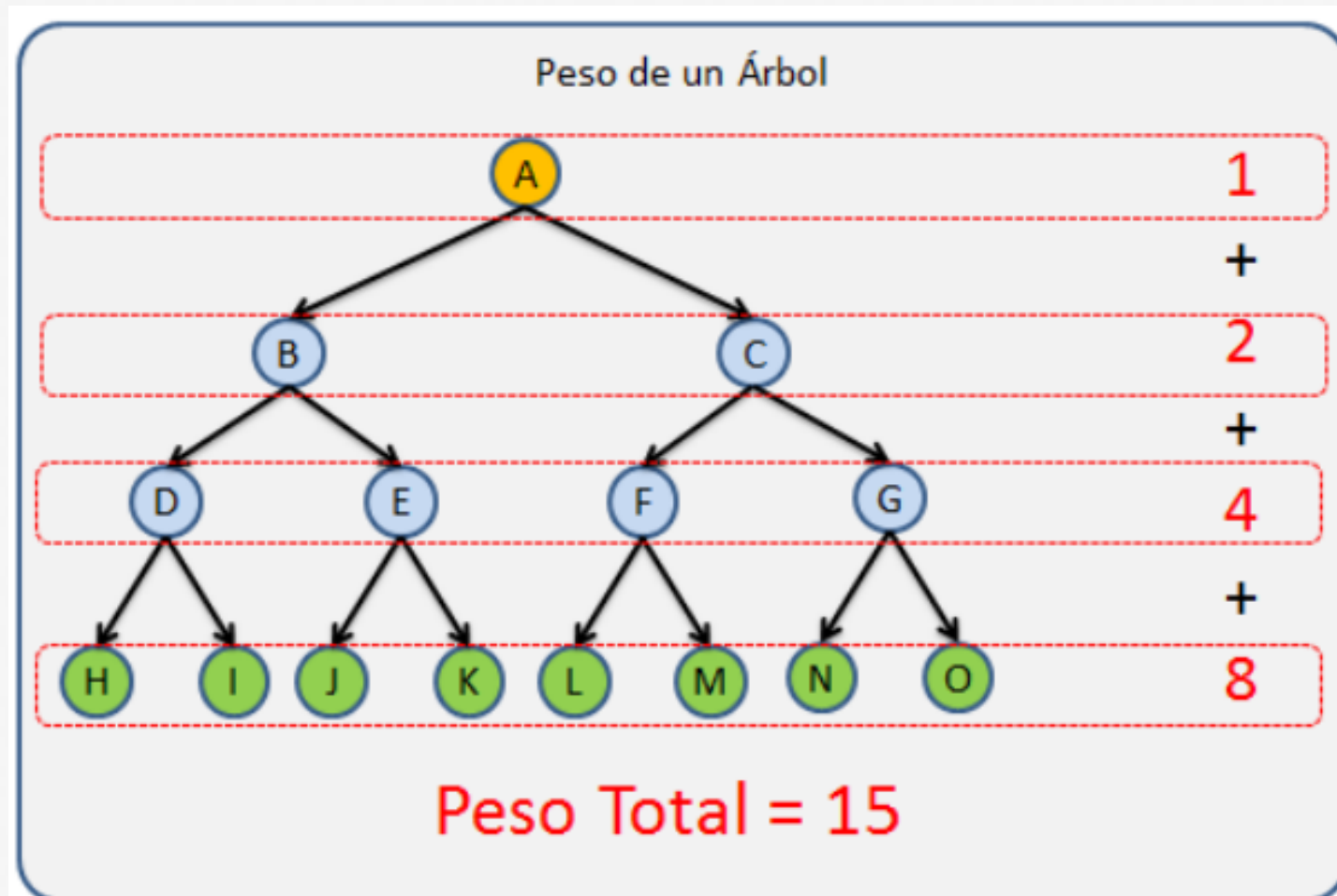
ÁRBOLES - tamaño

- **Altura:** Se le llama altura al número máximo de niveles de un árbol.



ÁRBOLES - tamaño

- *Peso*: Es el número total de nodos que tiene un árbol.

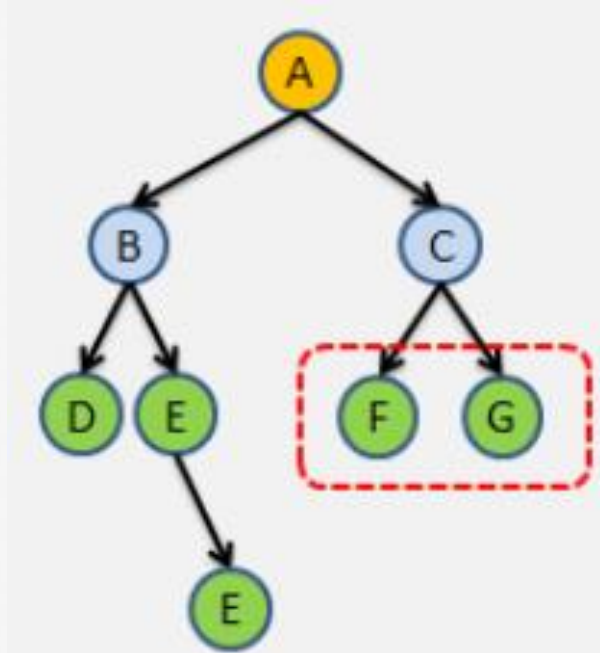


ÁRBOLES - tamaño

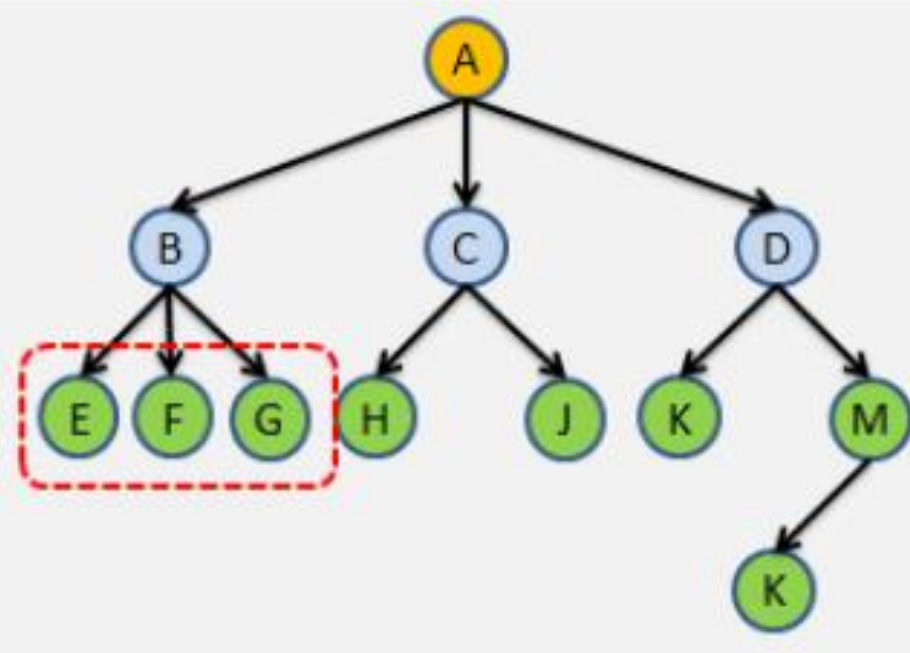
- **Grado:** El grado de un árbol es el número *máximo* de hijos que puede tener un Nodo. Es una *constante* que se define antes de crear el árbol.

Este valor no se calcula, si no que ya se conoce cuando se diseña la estructura.

ÁRBOL DE GRADO 2

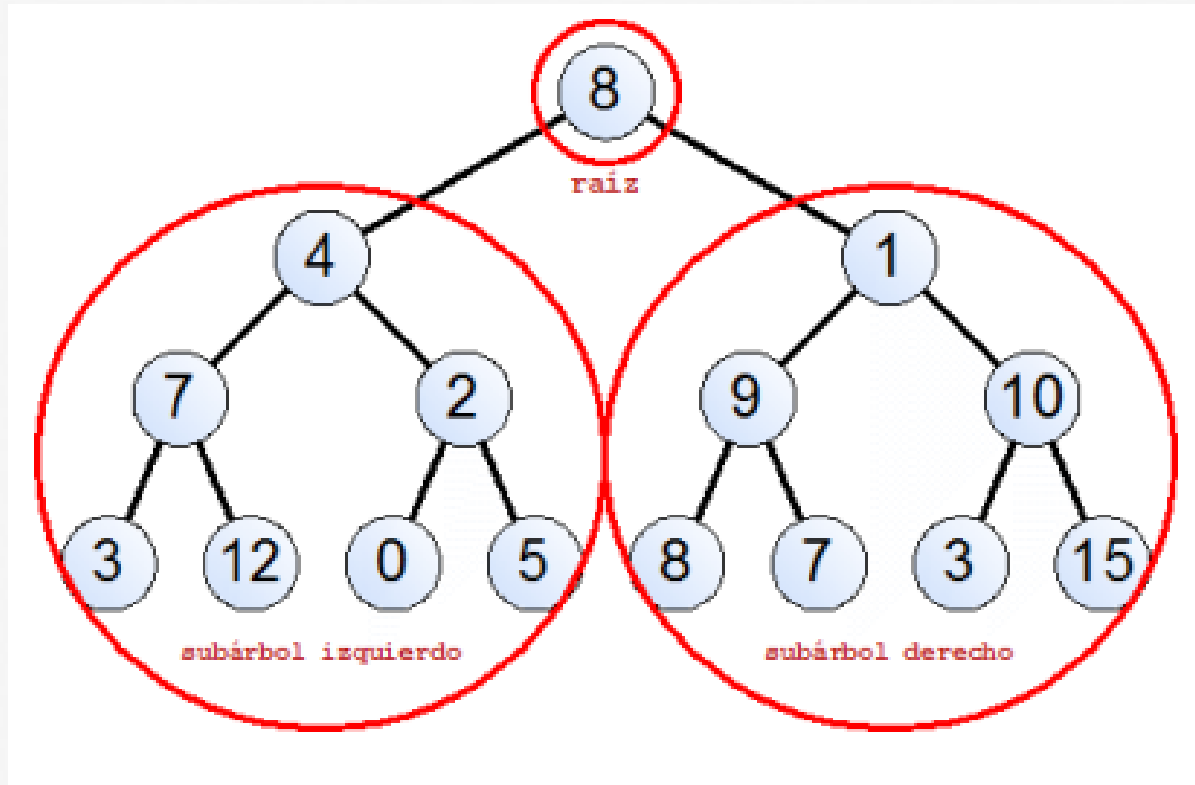


ÁRBOL DE GRADO 3



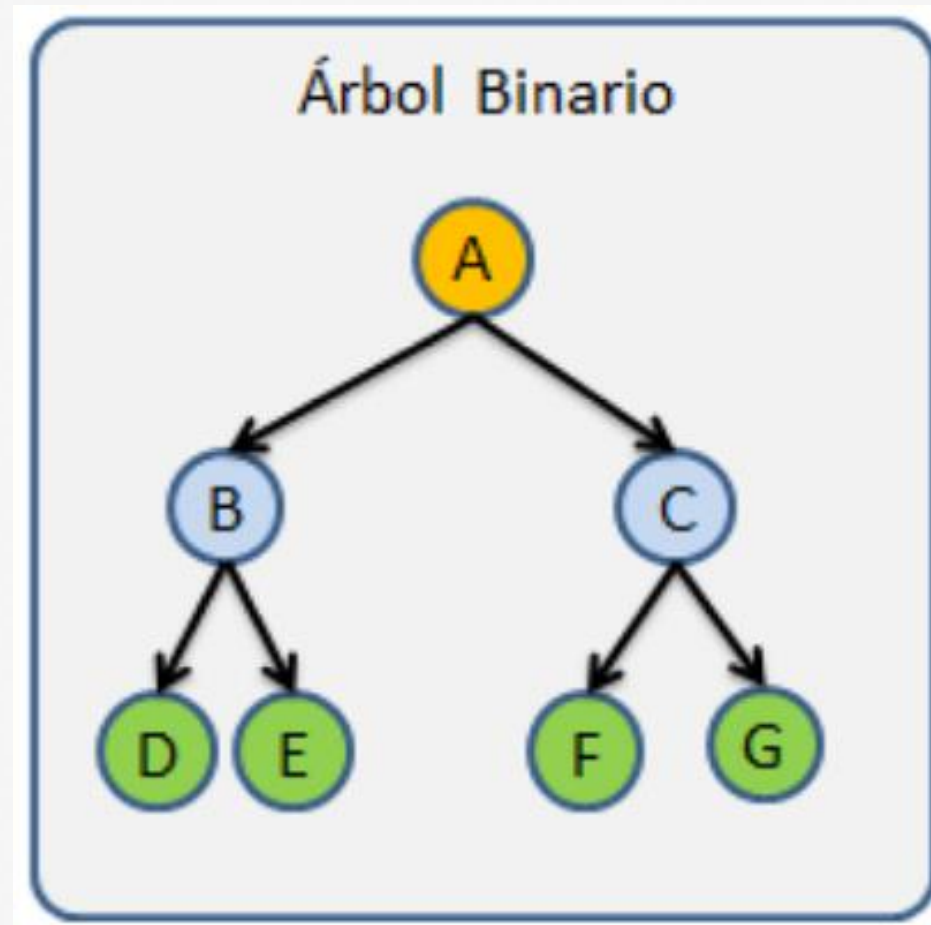
ÁRBOLES

- **Sub-Árbol:** Conocemos como Sub-Árbol a todo Árbol generado a partir de una sección determinada del Árbol, Por lo que podemos decir que un Árbol es un nodo Raíz con N Sub-Árboles.



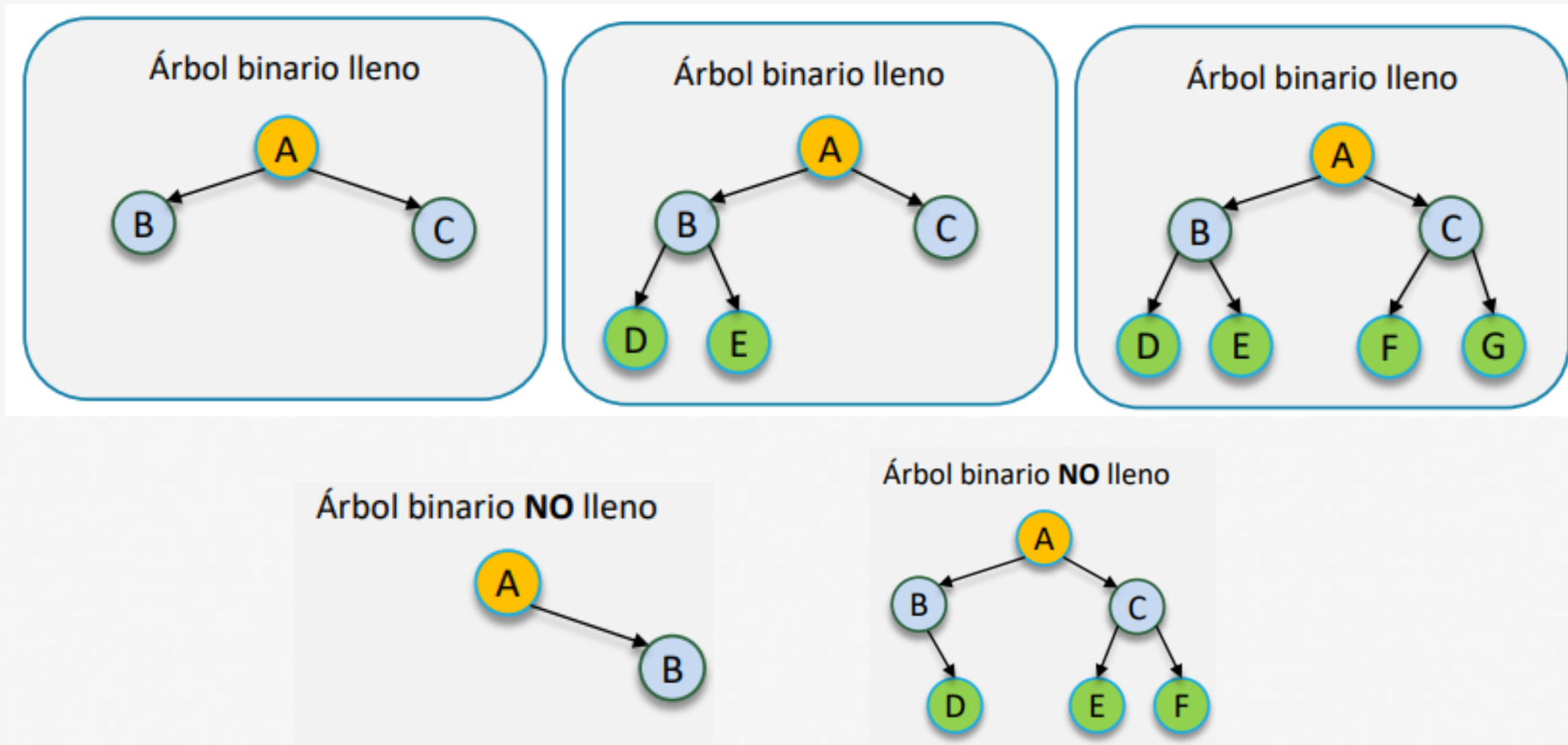
ÁRBOLES BINARIOS

Esta estructura se caracteriza por que cada nodo solo puede tener máximo 2 hijos, dicho de otra manera, es un árbol grado dos.



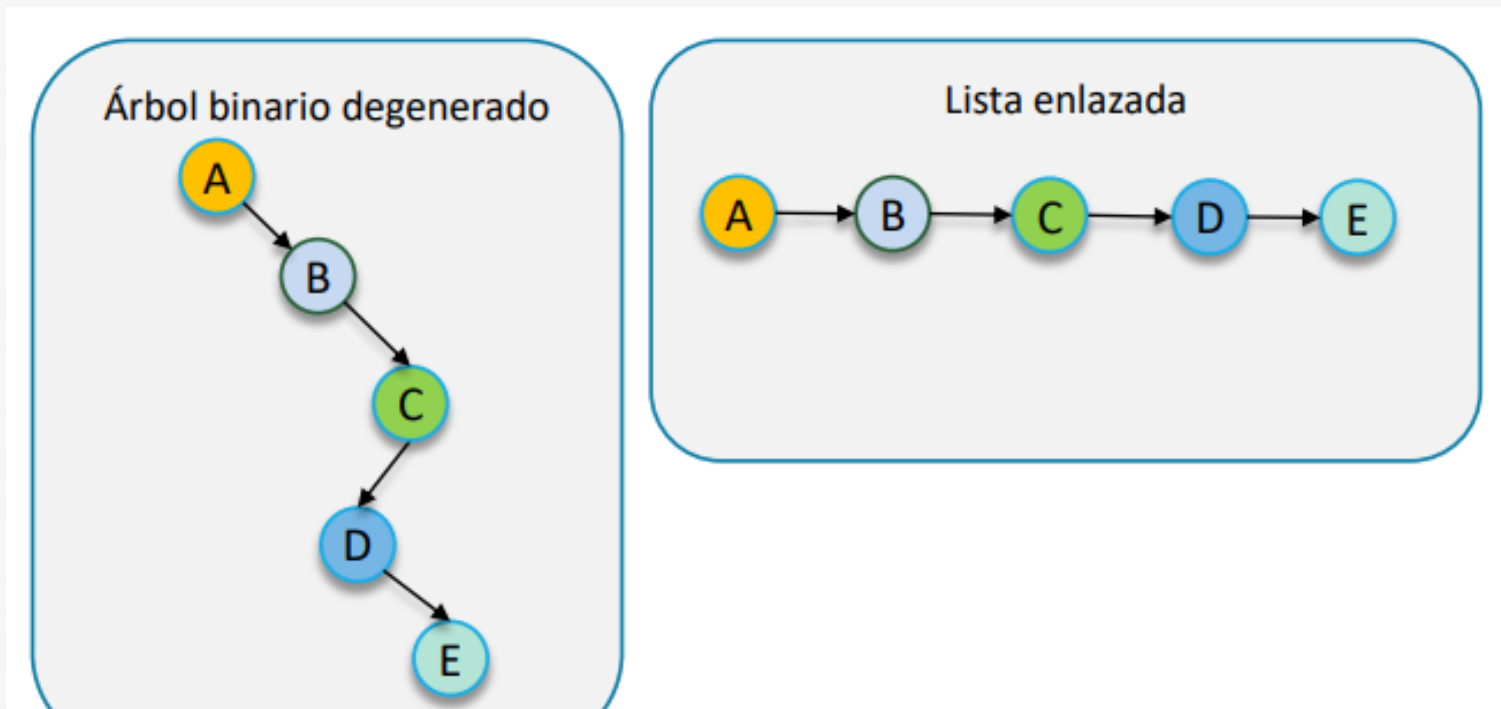
ÁRBOLES BINARIOS LLENOS

Es un árbol donde todos los nodos tienen cero o dos hijos. Es decir, no existe un nodo que tenga un solo hijo.



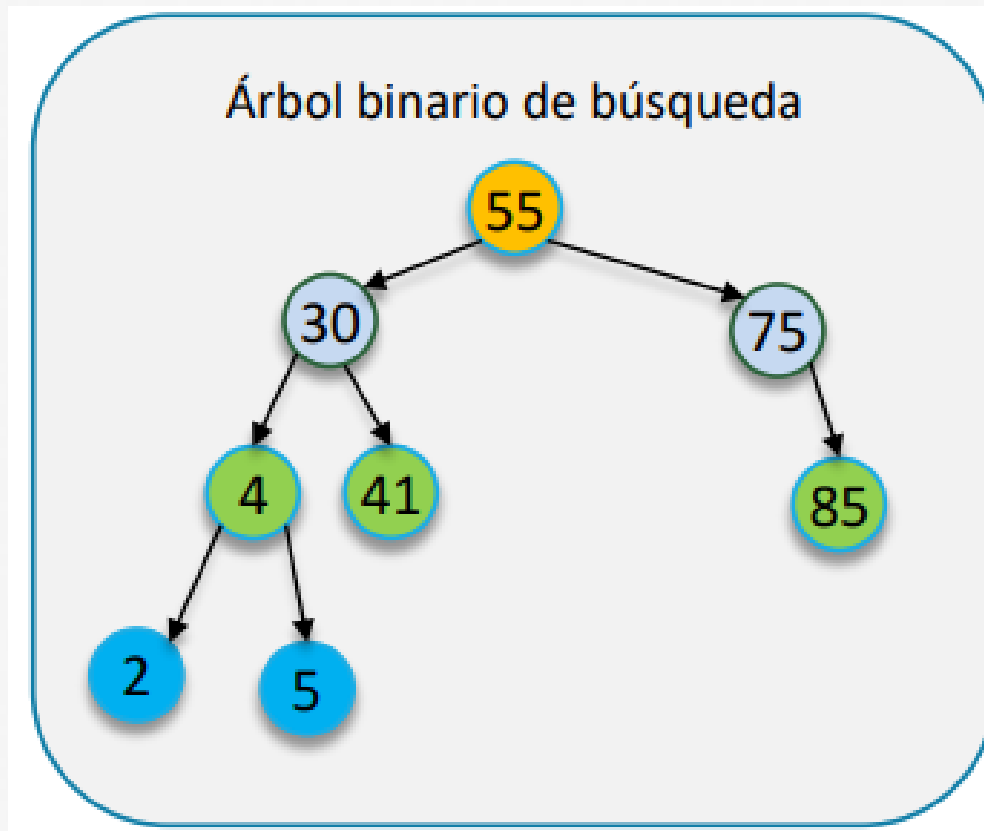
ÁRBOLES BINARIOS DEGENERADOS

Es un tipo especial de árbol en el que hay un solo nodo hoja y cada nodo rama sólo tiene un hijo. Un árbol degenerado es equivalente a una lista enlazada.



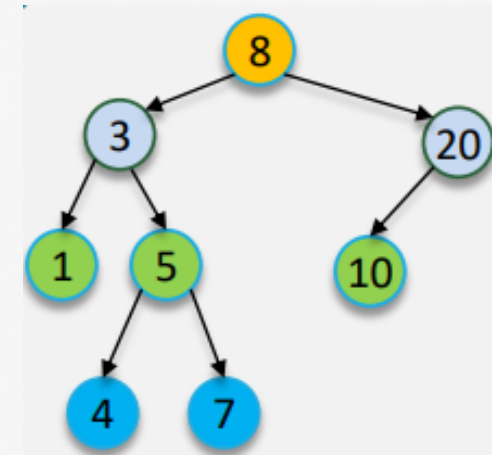
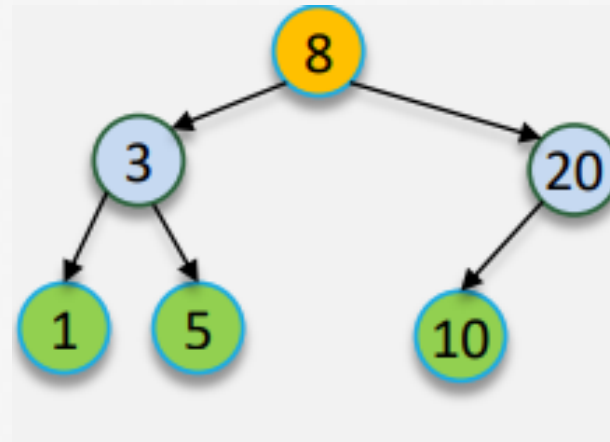
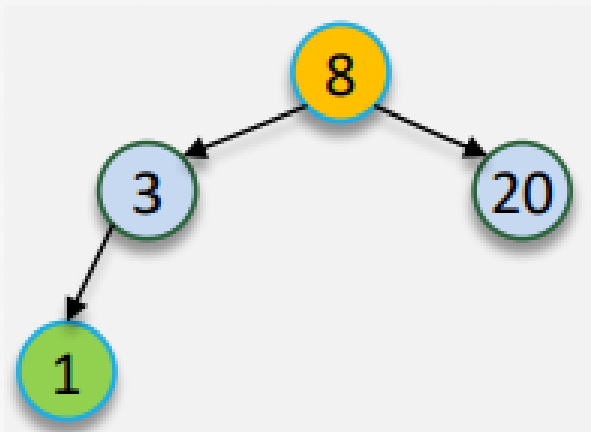
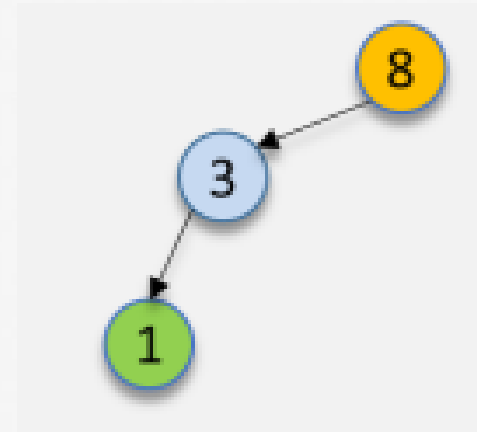
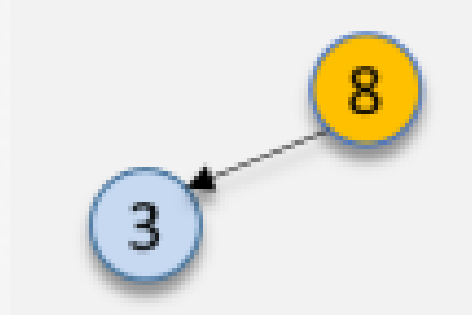
ÁRBOL BINARIO DE BÚSQUEDA

Un árbol binario de búsqueda es aquel en que, dado un nodo raíz, todos los datos del subárbol izquierdo son *menores* que los datos de esa raíz, mientras que todos los datos del subárbol derecho son *mayores*.



EJEMPLO

Graficar un árbol binario con los siguientes datos: 8, 3, 1, 20, 5, 10, 7, 4



ÁRBOL BINARIO DE BÚSQUEDA

Declaración de la clase nodo

```
class Nodo:  
    def __init__(self, dato):  
        self.dato = dato  
        self.izquierda = None  
        self.derecha = None
```

Inicialmente el nodo no tiene hijos

ÁRBOL BINARIO DE BÚSQUEDA

Definición de la clase árbol

```
class Arbol:  
  
    def __init__(self, dato):  
        self.raiz = Nodo(dato)
```

→ Para crear el árbol
debemos definir primero
la raíz

ÁRBOL BINARIO DE BÚSQUEDA

Insertar un nuevo dato:

Recordemos que primero definimos la raíz, y después todos los valores se agregan de acuerdo a ésta, ya que si el valor es menor lo ponemos en la rama izquierda, y si es mayor en la derecha.

En caso de que la rama ya esté ocupada: buscamos su subrama para, de nuevo, acomodar el valor. Es por eso que usamos recursión, ya que no se sabe cuándo se encontrará una rama disponible.

ÁRBOL BINARIO DE BÚSQUEDA

Insertar un nuevo elemento

```
class Arbol:

    def __init__(self, dato):
        self.raiz = Nodo(dato)

    def __agregar_recursivo(self, nodo, dato):
        if dato < nodo.dato:
            if nodo.izquierda is None:
                nodo.izquierda = Nodo(dato)
            else:
                self.__agregar_recursivo(nodo.izquierda, dato)
        else:
            if nodo.derecha is None:
                nodo.derecha = Nodo(dato)
            else:
                self.__agregar_recursivo(nodo.derecha, dato)
```

ÁRBOL BINARIO DE BÚSQUEDA

Insertar un nuevo elemento

```
class Arbol:

    def __init__(self, dato):
        self.raiz = Nodo(dato)

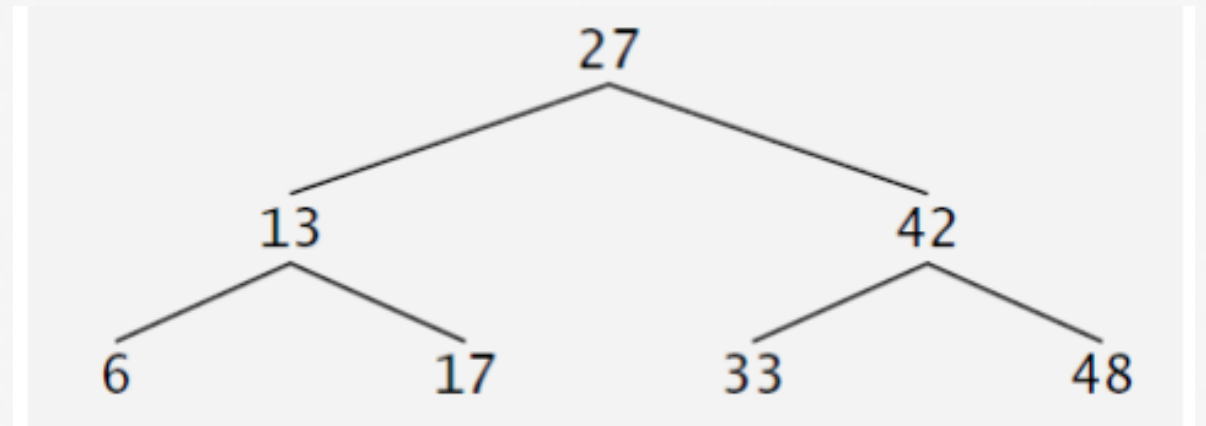
    def __agregar_recursivo(self, nodo, dato):
        if dato < nodo.dato:
            if nodo.izquierda is None:
                nodo.izquierda = Nodo(dato)
            else:
                self.__agregar_recursivo(nodo.izquierda, dato)
        else:
            if nodo.derecha is None:
                nodo.derecha = Nodo(dato)
            else:
                self.__agregar_recursivo(nodo.derecha, dato)
```

```
def agregar(self, dato):
    self.__agregar_recursivo(self.raiz, dato)
```

ÁRBOL BINARIO DE BÚSQUEDA

Crear un primer árbol

```
arbol_numeros = Arbol(27)
arbol_numeros.agregar(42)
arbol_numeros.agregar(13)
arbol_numeros.agregar(33)
arbol_numeros.agregar(17)
arbol_numeros.agregar(6)
arbol_numeros.agregar(48)
```



ÁRBOL BINARIO DE BÚSQUEDA

Recorrido de un árbol binario - **INORDEN**: *izquierdo, raíz, derecho*

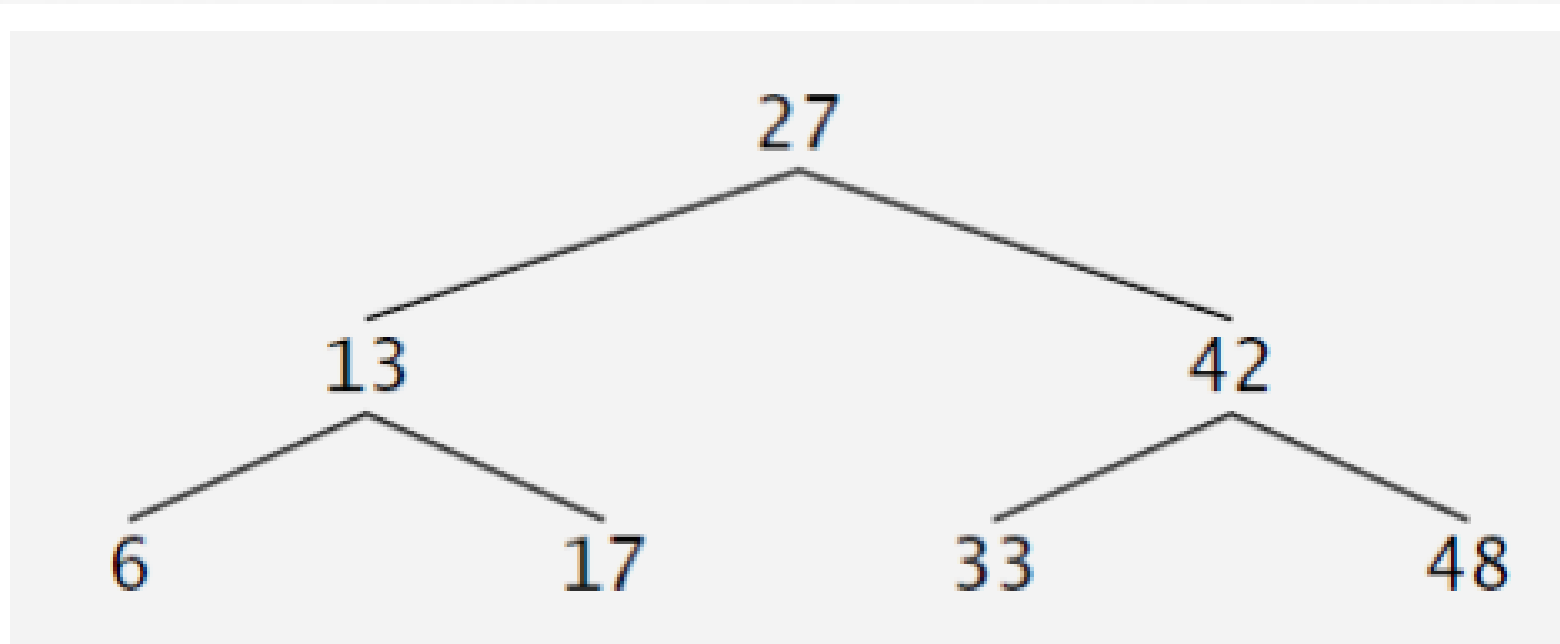
Para recorrer un árbol binario no vacío en InOrden, hay que realizar las siguientes operaciones comenzando con el nodo raíz:

- Recorrer el subárbol izquierdo InOrden
- Procesa la raíz
- Recorrer el subárbol derecho InOrden

De esta forma, se imprimen los valores en orden ascendente.

ÁRBOL BINARIO DE BÚSQUEDA

Recorrido de un árbol binario - INORDEN: *izquierdo, raíz, derecho*



Secuencia: 6 – 13 – 17 – 27 – 33 – 42 - 48

ÁRBOL BINARIO DE BÚSQUEDA

Recorrido de un árbol binario - INORDEN: *izquierdo, raíz, derecho*

```
def __inorden_recursivo(self, nodo):  
    if nodo is not None:  
        self.__inorden_recursivo(nodo.izquierda)  
        print(nodo.dato, end=", ")  
        self.__inorden_recursivo(nodo.derecha)
```

```
def inorden(self):  
    print("Imprimiendo árbol inorden: ")  
    self.__inorden_recursivo(self.raiz)  
    print("")
```

ÁRBOL BINARIO DE BÚSQUEDA

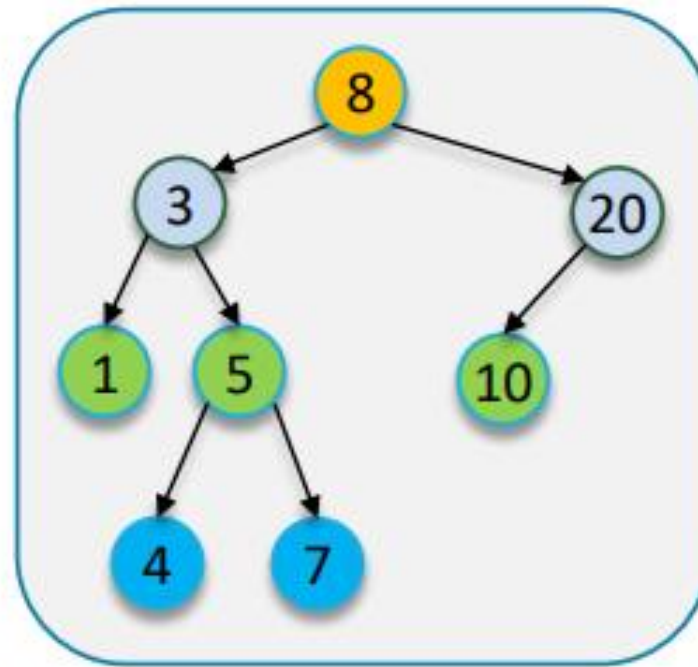
Recorrido de un árbol binario - INORDEN: *izquierdo, raíz, derecho*

```
arbol_numeros = Arbol(27)
arbol_numeros.agregar(42)
arbol_numeros.agregar(13)
arbol_numeros.agregar(33)
arbol_numeros.agregar(17)
arbol_numeros.agregar(6)
arbol_numeros.agregar(48)

arbol_numeros.inorden()
```

ACTIVIDAD

Programar el siguiente árbol binario e imprimir sus elementos InOrden



Secuencia: 1 – 3 – 4 – 5 – 7 – 8 – 10 – 20

ÁRBOL BINARIO DE BÚSQUEDA

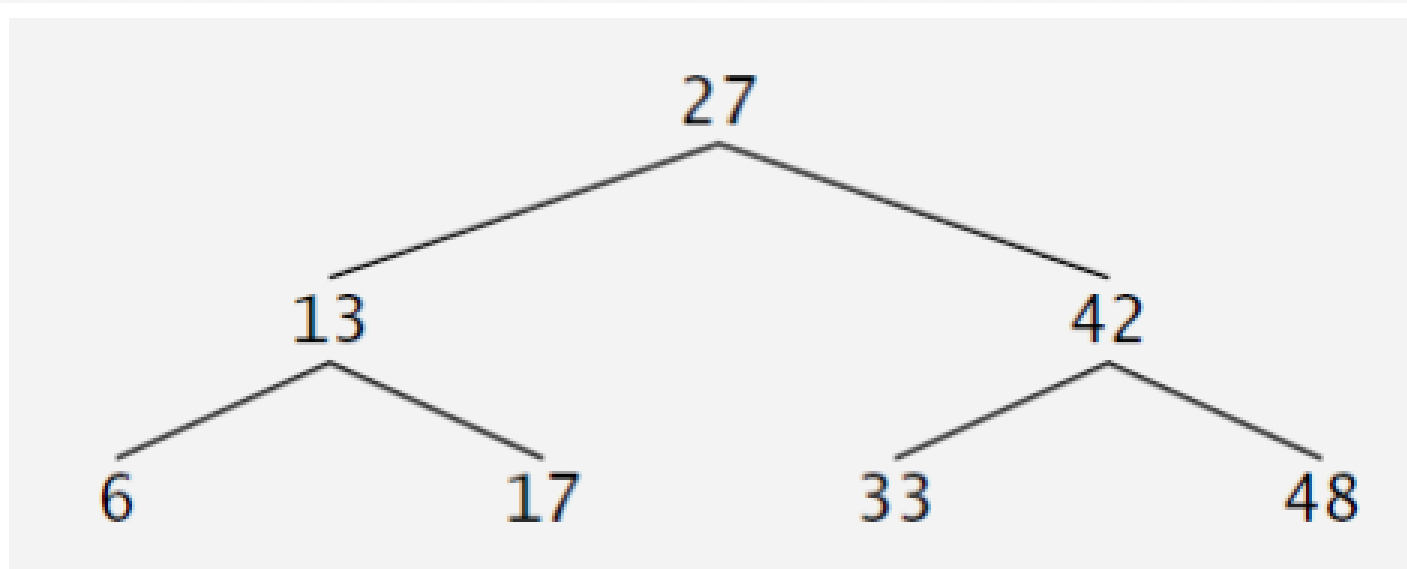
Recorrido de un árbol binario - PREORDEN: *raíz, izquierdo, derecho*

Para recorrer un árbol binario no vacío en PreOrden, hay que realizar las siguientes comenzando con el nodo raíz:

- Procesa la raíz
- Recorrer el subárbol izquierdo en PreOrden
- Recorrer el subárbol derecho en PreOrden

ÁRBOL BINARIO DE BÚSQUEDA

Recorrido de un árbol binario - PREORDEN: *raíz, izquierdo, derecho*



Secuencia: 27 – 13 – 6 – 17 – 42 – 33 – 48

ÁRBOL BINARIO DE BÚSQUEDA

Recorrido de un árbol binario - PREORDEN: *raíz, izquierdo, derecho*

```
def __preorden_recursivo(self, nodo):  
    if nodo is not None:  
        print(nodo.dato, end=" ", "  
        self.__preorden_recursivo(nodo.izquierda)  
        self.__preorden_recursivo(nodo.derecha)
```

```
def preorden(self):  
    print("Imprimiendo árbol preorden: ")  
    self.__preorden_recursivo(self.raiz)  
    print("")
```

ÁRBOL BINARIO DE BÚSQUEDA

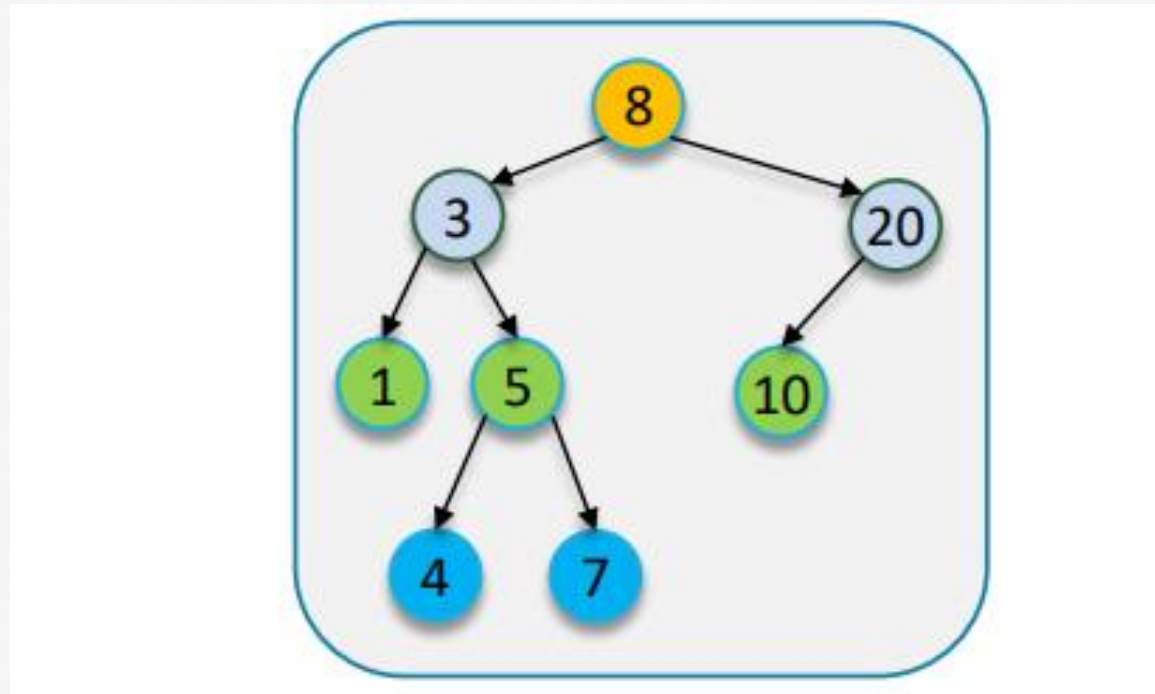
Recorrido de un árbol binario - PREORDEN: *raíz, izquierdo, derecho*

```
arbol_numeros = Arbol(27)
arbol_numeros.agregar(42)
arbol_numeros.agregar(13)
arbol_numeros.agregar(33)
arbol_numeros.agregar(17)
arbol_numeros.agregar(6)
arbol_numeros.agregar(48)

arbol_numeros.preorden()
```

ACTIVIDAD

Programar el siguiente árbol binario e imprimir sus elementos PreOrden



Secuencia: 8 – 3 – 1 – 5 – 4 – 7 – 20 – 10

ÁRBOL BINARIO DE BÚSQUEDA

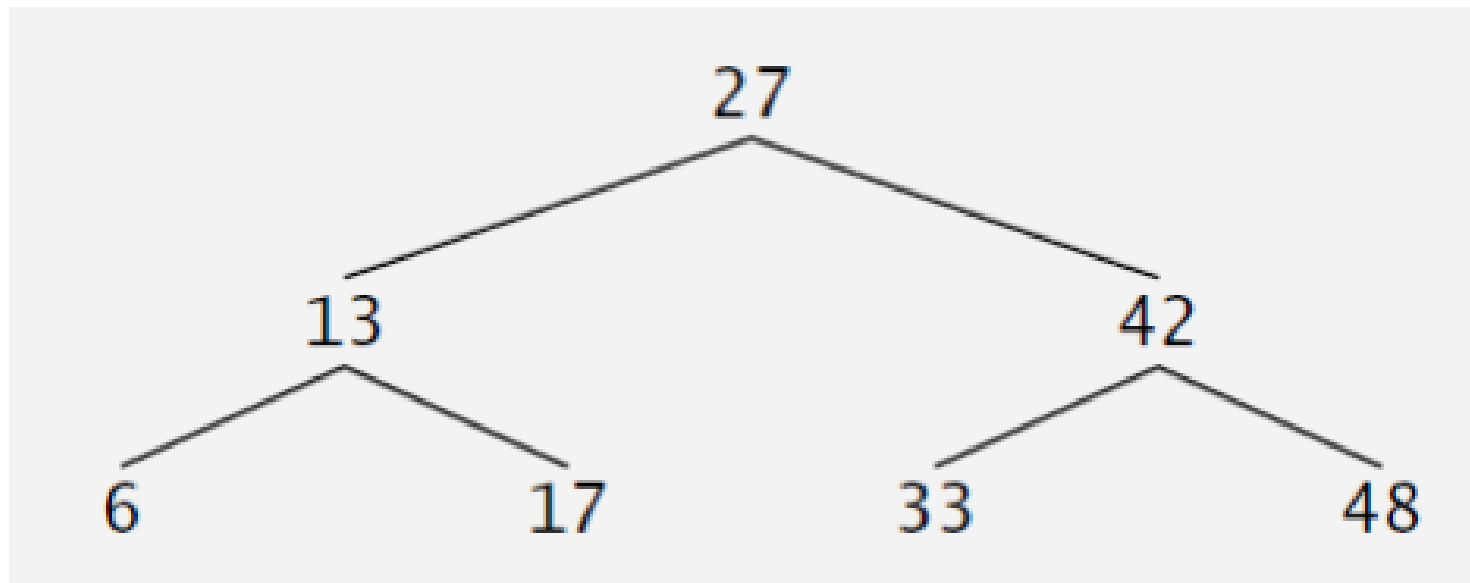
Recorrido de un árbol binario - POSTORDEN: *izquierdo, derecho, raíz*

Para recorrer un árbol binario no vacío en PostOrden, hay que realizar las siguientes operaciones comenzando con el nodo raíz:

- Recorrer el subárbol izquierdo en PostOrden
- Recorrer el subárbol derecho en PostOrden
- Procesa la raíz

ÁRBOL BINARIO DE BÚSQUEDA

Recorrido de un árbol binario - POSTORDEN: *izquierdo, derecho, raíz*



Secuencia: 6 – 17 – 13 – 33 – 48 – 42 - 27

ÁRBOL BINARIO DE BÚSQUEDA

Recorrido de un árbol binario - POSTORDEN: *izquierdo, derecho, raíz*

```
def __postorden_recursivo(self, nodo):  
    if nodo is not None:  
        self.__postorden_recursivo(nodo.izquierda)  
        self.__postorden_recursivo(nodo.derecha)  
        print(nodo.dato, end=", ")
```

```
def postorden(self):  
    print("Imprimiendo árbol postorden: ")  
    self.__postorden_recursivo(self.raiz)  
    print("")
```

ÁRBOL BINARIO DE BÚSQUEDA

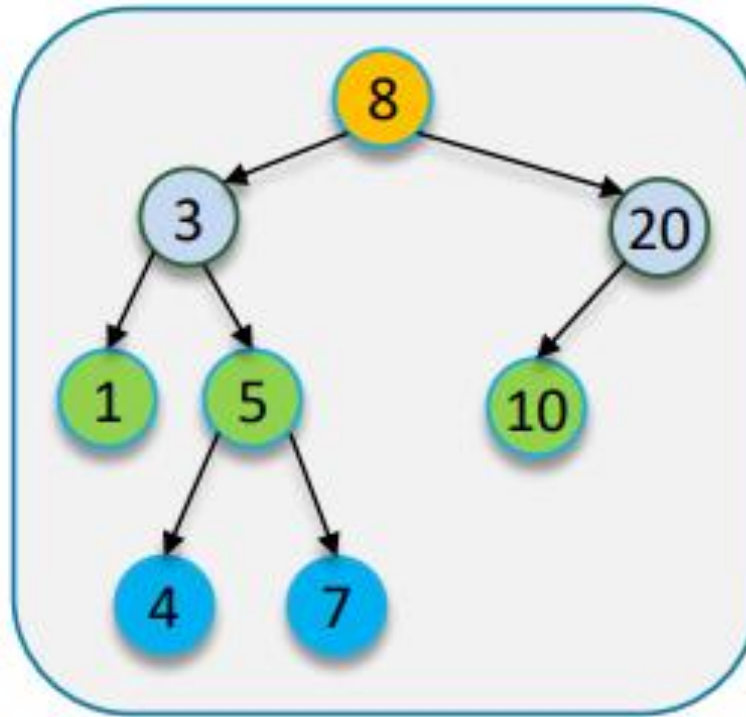
Recorrido de un árbol binario - POSTORDEN: *izquierdo, derecho, raíz*

```
arbol_numeros = Arbol(27)
arbol_numeros.agregar(42)
arbol_numeros.agregar(13)
arbol_numeros.agregar(33)
arbol_numeros.agregar(17)
arbol_numeros.agregar(6)
arbol_numeros.agregar(48)

arbol_numeros.postorden()
```


ACTIVIDAD

Programar el siguiente árbol binario e imprimir sus elementos PostOrden



Secuencia: 1 – 4 – 7 – 5 – 3 – 10 – 20 – 8

ÁRBOL BINARIO DE BÚSQUEDA

Buscar un elemento dentro de un árbol

```
def __buscar(self, nodo, busqueda):  
    if nodo is None:  
        return None  
    if nodo.dato == busqueda:  
        return nodo  
    if busqueda < nodo.dato:  
        return self.__buscar(nodo.izquierda, busqueda)  
    else:  
        return self.__buscar(nodo.derecha, busqueda)
```

```
def buscar(self, busqueda):  
    return self.__buscar(self.raiz, busqueda)
```

ÁRBOL BINARIO DE BÚSQUEDA

Buscar un elemento dentro de un árbol

```
arbol_numeros = Arbol(27)
arbol_numeros.agregar(42)
arbol_numeros.agregar(13)
arbol_numeros.agregar(33)
arbol_numeros.agregar(17)
arbol_numeros.agregar(6)
arbol_numeros.agregar(48)

busqueda = int(input("Ingresa un número para buscar en el árbol: "))
n = arbol_numeros.buscar(busqueda)
if n is None:
    print(f"{busqueda} no existe")
else:
    print(f"{busqueda} sí existe")
```

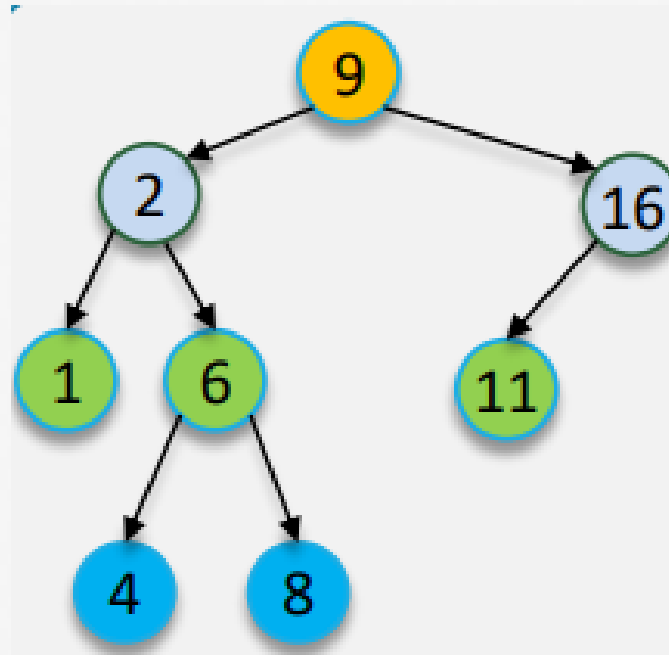
ACTIVIDAD

1) Programar un árbol binario con los siguientes datos: 9, 2, 1, 16, 6, 11, 8, 4

- pruebe todos los métodos programados anteriormente

2) Ayuda para el parcial

- consulte: un método para calcular la altura del árbol y un método para calcular el peso del árbol





Institución
Universitaria
Reacreditada en Alta Calidad

¡MUCHAS GRACIAS!

A decorative graphic consisting of several horizontal lines in various colors (yellow, orange, red, purple, blue, green) and a stylized white 'C' shape on the right side.